Title: Student Attendance Management System

Problem Statement: Manual attendance tracking in educational institutions is often inefficient, human error and lacks real-time accessibility. Educators frequently struggle with maintaining physical registers or static spreadsheets that do not provide instant analytics on student participation. The proposed system addresses these issues by utilizing a structured object-oriented hierarchy to model students and attendance records. By automating the calculation of attendance percentages and providing instant report generation, the system ensures data integrity, reduces administrative workload, and offers clear visibility into student performance trends.

Target User: To add students to the database and perform a daily "roll call" through the console. To view comprehensive attendance reports and identify students with low participation.

Core Features:

- **Student Record Management:** Add, update, and delete student profiles with unique identifiers.
- **Interactive Attendance Logging:** A digital interface to mark students as present or absent for daily sessions.
- **Real-Time Analytics:** Automatic calculation of attendance percentages for every student.
- **Consolidated Report Generation:** View comprehensive attendance summaries for the entire class.
- **Persistent Data Storage:** Ability to save and load student records for long-term tracking.
- **Robust Exception Handling:** Validation for user inputs and data processing errors.

OOP Concepts to be Used:

- **Abstraction:** Abstract Person class defining shared behaviors and attributes like id and name.
- **Inheritance:** Specialized Student subclass extending the Person class to include attendance-specific data.
- **Polymorphism:** Overridden displayInfo() methods to provide custom data representations for different user types.
- **Exception Handling:** Managing invalid inputs (e.g., non-numeric menu choices) and file-related errors.
- **Collections:** ArrayList or HashMap for dynamic management and retrieval of student objects.

Architecture: The project follows a **Controller-Model design pattern**. The Student and Person classes act as the **Models** representing the data. The AttendanceManager acts as the **Controller**, containing the logic for adding students and calculating records. The Main class serves as the **View/Interface**, handling the user's input and output via the Scanner class. This modular approach makes the code easy to maintain and scale.